

RedByte Project Strategic Plan

RedByte is envisioned as a **browser-based OS** for digital logic education and FPGA lab automation. Its core is a visual logic circuit simulator: students **build digital circuits and simulate them in real time in the browser** ¹. Underlying RedByte is a modern monorepo with a modular architecture: packages include a pure logic simulation engine (`rb-logic-core`), 2D/3D circuit renderers, a desktop shell (`rb-shell`), and built-in applications for the logic playground and courses ². This design ensures a unified platform where any FPGA circuit or chip design can be authored, simulated, and exported in standard formats. Current components in development include:

- **Runtime Bridge** (`rb-fpga-bridge`): a local HTTP/WS service for board discovery, programming, and streaming I/O.
- **Build Toolchain** (`rb-fpga-toolchain`): orchestration of synthesis and programming (Vivado, ISE, or open-source tools) with automatic wrapper generation.
- **Board Models**: JSON/XDC/UCf schemas defining supported FPGA boards (today Basys 3 and Spartan-3E) and their pin mappings.
- **Wrapper & Protocol Layers**: code (`wrapper.js`, etc.) that wraps user HDL with standard I/O and enforces a deterministic data-stream protocol.
- **Smoke Test Pipeline**: an automated PowerShell test suite verifying end-to-end lab readiness (device detection, programming, streaming, etc.).
- **Lab Configuration**: per-lab `lab.json` files (listing board IDs, top-module names, expected I/O) guiding the toolchain and grading.
- **Evidence & Contract Tests**: ongoing work to compare student submissions against “golden” baselines, producing automated feedback.

In summary, RedByte aims to replace siloed proprietary tools with an integrated **universal simulator and education platform**, deep-linking circuit design, simulation, and real hardware feedback.

Architecture & Core Components

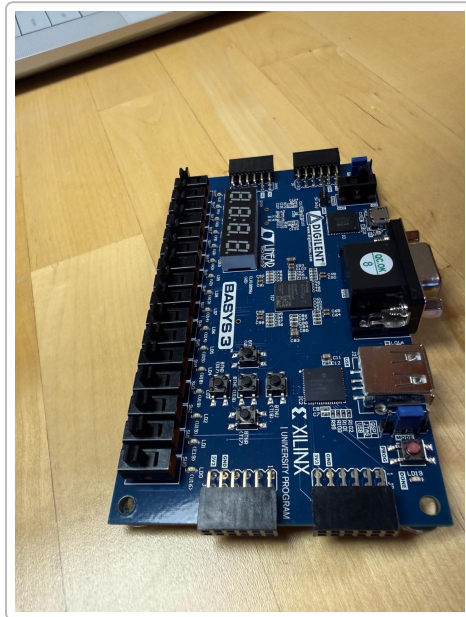
RedByte’s **monorepo architecture** enforces a clean separation of concerns. The *Logic Core* handles gate-level simulation, while *view* packages render schematics and 3D layouts. A desktop “shell” provides an OS-like environment with windows, a launcher, and apps (e.g. a logic playground, tutorials, etc.) ². For example, the documentation describes RedByte as “a visual logic circuit simulator...in your browser” ¹, with **zero-setup** usability and built-in example circuits. Under the hood, any user circuit can be exported as Verilog or bitstream; conversely, existing FPGA designs can be imported into the simulator. The shell and apps are built with TypeScript/React (using pnpm workspaces and Vite), and the entire web UI is auto-deployed via Cloudflare Pages on each main-branch update ³.

Key packages in the repo include `@redbyte/rb-logic-core` (simulation engine), `@redbyte/rb-logic-view/3d` (rendering), `@redbyte/rb-shell` (the desktop environment), and `@redbyte/rb-apps` (interactive applications) ². The architecture enforces contracts (like window-focus rules and intent handling) to ensure a consistent user experience. Crucially, the **rb-fpga-toolchain** package provides the glue from HDL to bitstream: it automatically generates a standard wrapper (with fixed I/O

names) around the student's top module, writes toolchain TCL scripts, and launches Vivado or ISE to produce a `.bit` file. It also supports open-source flows (openFPGALoader) as fallbacks. The toolchain outputs a **manifest JSON** for each build, capturing design hashes, wrapper version, pinmap hash, build status, and artifact names, enabling reproducible builds and later comparison ⁴ ⁵.

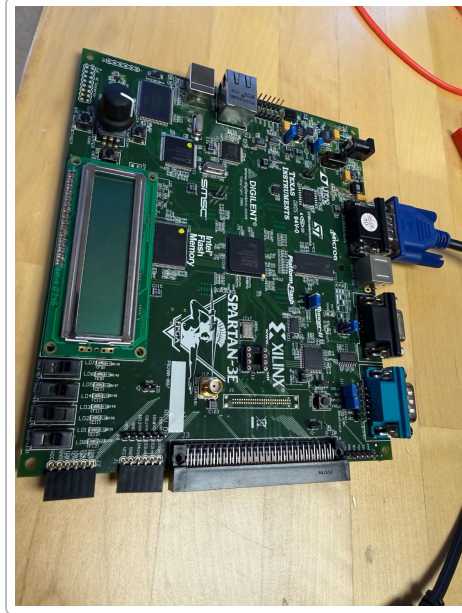
FPGA Hardware Integration

RedByte tightly integrates with real FPGA boards for hands-on labs. A local **FPGA Bridge** service (`rb-fpga-bridge`) communicates with connected Digilent boards over USB. It merges UART and JTAG channels into a unified device API: for example, a `/devices` query lists Basys3 or Spartan-3E boards and their runtime/programming status. The smoke-test script proves this end-to-end pipeline: it programs a test bitstream to the board and starts a hardware run, then streams real samples via Server-Sent Events, verifying signals are received ⁶.



Basys 3 FPGA board (Xilinx Artix-7) used in RedByte labs. RedByte's board-model schema currently supports the Basys3 and Spartan-3E (shown below) platforms ⁷. The bridge service programs and streams from these boards over USB.

The **Board Models** define each supported FPGA platform. In code, the models are declared with IDs and pinmaps, e.g. `"basys3"` → *Basys 3* (Artix-7, with a Vivado XDC pin mapping) and `"spartan3e-starter"` → *Spartan-3E Starter Kit* (with an ISE UCF) ⁷. These schemas ensure the wrapper generator and bridge know each board's LED/switch/serial pins and voltage standards. Adding new hardware simply means adding a new model with its pins and JTAG vendor IDs; the bridge's discovery logic will then detect matching USB VID/PIDs and allow programming. For example, the Basys3 photo above is recognized via its Digilent USB IDs and exposes a `/stream` of logic outputs.



Spartan-3E Starter Kit (Xilinx Spartan-3E) – RedByte’s second supported board. The fpga-bridge detects this board by USB/JTAG, programs the student bitstream, and streams out sample events for analysis. Supporting more boards involves extending the `BOARD_MODELS` definitions and (if needed) adding open-source loader support.

All FPGA interactions are deterministic: the wrapper enforces a fixed protocol (named `rb_protocol`) and fixed clock rates, so the same input circuit always yields the same output stream barring hardware faults. The **wrapper generator** (`generateWrapperVerilog`) produces Verilog with a standard interface (e.g. `input clk`, `output[7:0] leds`) and can be version-controlled (the manifest tracks `WRAPPER_VERSION`). This ensures that lab evaluations are reproducible: for instance, the smoke test compares received vectors to a golden baseline, and a regression script (`check-golden.js`) can diff new output capsules against the blessed ones to catch unintended changes ⁸.

Toolchain & Testing Automation

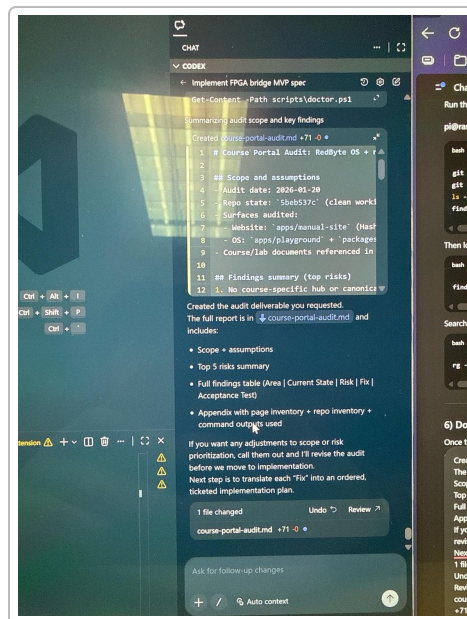
Building a student circuit involves multi-step automation. The `rb-fpga-toolchain` provides a CLI (`rb-fpga-toolchain build --board <name> --src <dir> --top <module> --out <dir> [--lab <lab.json>]`) that:

1. **Collects HDL sources:** scans the source directory for `.v` / `.sv` files.
2. **Generates a standard wrapper:** calls `buildSampleTemplate` to inject the student’s top module into a testbench shell with LEDs, switches, etc.
3. **Checks interface compliance:** compares the student’s top I/O against the lab’s expected interface (if a `lab.json` specifies required signals).
4. **Writes synthesis scripts:** for Vivado (or for ISE on Spartan), writing TCL to read the wrapper, sources, and the board’s pinmap, then run `synth/place/route` to generate `bitstream.bit`.
5. **Runs the tool:** if Vivado is available, it is launched in batch mode; otherwise, the task is skipped (the manifest will note “skipped” status).
6. **Produces artifacts:** on success it writes the bitstream plus logs (synthesis reports, manifest, etc.) to the output directory. The manifest JSON records everything (toolchain version, status, build ID, bitstream hash, etc.) ⁴ ⁵.

A **contract test** suite is under development: each lab can have a set of test vectors or expected behaviors. RedByte’s smoke pipeline (a week-1 sanity test) already verifies that the board responds on UART as expected ⁶. In future phases, each lab submission could be automatically run through an **agent-driven test harness**: for example, programming the student bitstream, simulating inputs, and comparing outputs against contract tests. Scripts like `check-golden.js` act as regression gates: they diff the new output capsule against a golden baseline, failing if any unexpected bit flips occur ⁸. Together, these contract tests and evidence generators will provide automated student feedback (“Your circuit failed test #3: LED1 did not toggle”).

Antigravity Agent Orchestration

A key innovation is using **Google Antigravity’s agentic IDE platform** to automate development and feedback tasks. Antigravity introduces a dual interface: a standard AI-enhanced code editor **plus** an “Agent Manager” surface for *asynchronous, multi-agent workflows* ⁹ ¹⁰. In practical terms, RedByte developers (and ultimately instructors) can **spawn AI agents** that autonomously plan, execute, and verify complex tasks across code, terminal, and browser ⁹ ¹¹. For example, one agent could “Generate test benches for all labs,” simultaneously opening editors to scaffold code, invoking the toolchain to build bitstreams, running the bridge to program devices, and then collecting output logs — all without interrupting the user.



Example Agent Workflow: An Antigravity agent summarizes the RedByte codebase into an audit report. The agent-first interface (left) runs alongside the code editor (right). Artifacts like the generated `course-portal-audit.md` (bottom) let developers review what the agent produced. This illustrates how Antigravity’s Manager surface can dispatch tasks (e.g. code audits, test generation) and generate reviewable artifacts automatically.

RedByte can exploit these capabilities in several ways:

- **Task Delegation:** Delegate entire tasks to agents. For instance, “Build and verify lab 2 on Basys3” can be a single instruction. An agent will write the necessary synthesis TCL, launch Vivado, program the FPGA via the bridge, and run the smoke test. When complete, it reports back. This frees developers from manual context-switching ¹¹.

- **UI-driven Iteration:** Agents can perform visual/functional updates. E.g., “Add a dark-mode toggle to the website.” The agent would modify CSS/JS, rebuild the site, then provide **Artifacts** (screenshots, video clips, diff reports) showing the new UI. RedByte can adopt this to prototype interactive tools (logic editors, tutorial slides) with AI assistance ¹².
- **Asynchronous Maintenance:** Long-running tasks like “audit all student lab submissions” or “generate testbench suite” can run in the background. The Agent Manager lets multiple agents run in parallel – one agent per lab or per board, for example – and developers monitor progress via high-level artifacts rather than raw logs. Antigravity’s Artifacts system (screenshots, recordings, execution logs, test results) ensures trust: instead of reading a wall of console text, a developer gets a clear proof-of-work to review ¹³.
- **Knowledge Reuse:** Antigravity agents persist learnings in a knowledge base. Over time, as agents build more wrapper templates or solve repeated synthesis errors, those patterns are cached. Future tasks (like adding a new board) become easier because agents remember working solutions.

By integrating Antigravity, RedByte turns agent use *inside the product development loop*. Agents could even assist students: for example, a “homework helper agent” that reads a student’s circuit, runs it against sample inputs, and points out logical errors (acting as an intelligent tutor). Within the RedByte IDE, students might use agents to refactor their HDL or auto-generate test vectors.

AI Collaboration & Models

RedByte’s agent ecosystem isn’t limited to Antigravity’s UI: it can leverage **multiple LLM backends** for maximum flexibility. Google’s platform natively supports Gemini Pro, Claude, and even OpenAI’s GPT models (“GPT-OSS”) ¹⁴. This means RedByte can blend models: use Claude for fast code edits, ChatGPT (via GPT-OSS or API) for explanatory feedback, and Gemini for planning tasks. For instance, an agent could ask ChatGPT to “explain why my circuit output is wrong” by feeding it waveforms or HDL snippets, then incorporate that insight into an action plan.

Antigravity itself abstracts the model details: to the RedByte team and students, the interface is the same “Agent”. But behind the scenes, an agent might call out to ChatGPT for natural-language responses, to Gemini for coding work, or to Claude for summarization. This hybrid approach closes the loop from **IDE to hardware and back**. For example, after programming a board, an agent could summarize the results (“LED2 blinked once instead of twice”) and use that in formulating follow-up tasks or hints. Agents could even generate new labs or hints based on observed student weaknesses, effectively combining the broad knowledge of ChatGPT with RedByte’s domain-specific checkers.

Website & Interface Evolution

RedByte’s public face (e.g. redbyteapps.dev) should evolve to showcase these capabilities. The current site (see its homepage content ¹⁵) emphasizes the logic simulator and tutorials. Going forward, it should also highlight RedByte’s FPGA integration and AI features. Suggestions include:

- **Interactive Diagrams:** Add architecture diagrams showing how designs flow from the browser to FPGAs and back, possibly with animated agents reviewing code.
- **Lab Portals:** Build a “Courses” portal where instructors can create labs and students can upload solutions. This portal would integrate with RedByte’s backend to automatically run tests and present feedback. (This is hinted in [27†L31-L39], where stable demo artifacts are linked.)
- **Agent Demos:** Include demos of Antigravity in action (e.g., video screencasts or live sessions showing

agents auditing code or optimizing circuits). This will convey the “agent-first” philosophy to the community.

- **Enhanced UI Apps:** Revamp the logic playground to incorporate new views (e.g. waveform timeline, logic analyzer). In the FPGA Proof Viewer app, add polish features like “Download Report” or “Share Demo” as outlined in the production checklist ¹⁶.

- **Real-time Interaction:** Through WebRTC or polling, allow students to see their hardware running in an embedded viewer, reinforcing the IDE-to-hardware loop.

Under the hood, the site should continue using Cloudflare Pages auto-deploys and include AI-driven static-generation where appropriate (e.g. pre-building tutorial content via agent scripts). All new pages should be fully linked from the launcher (Explorer > Courses, FPGA Labs, etc.) so the UI matches the expanding feature set.

Scaling Hardware Support

To **scale beyond two boards** while keeping everything deterministic, RedByte will leverage its existing abstractions. The `board-models` package can be extended with new board IDs and pinmaps (for example, Nexys A7, Arty, etc.). Because the toolchain uses BoardModels and openFPGALoader, adding a new board often just means adding its XDC/UCF and enabling support in the CLI. The fpga-bridge’s discovery (USB VID/PID + JTAG detection) will detect any Digilent- or FTDI-based kit automatically using the same scoring logic ⁷. We will verify new boards in the smoke pipeline (modifying `-Board` flags), ensuring the same protocol semantics.

Staying deterministic requires tight control over the toolchain. All builds should use fixed versions of Vivado/ISE or containerized open-source flows, and the wrapper protocol must remain backwards-compatible. We will maintain a **golden reference library**: for each supported board and lab, we store a golden output stream (capsule) and test suite. Future agent-based CI pipelines will compare new student submissions and even updated RedByte code against these references. This way, adding hardware doesn’t introduce nondeterminism. If necessary, we can isolate non-determinism by switching to purely simulated I/O for new boards (where reproducibility is easier) before full hardware rollout.

Development & Deployment Roadmap

A phased strategy will guide RedByte’s evolution:

1. **Solidify Core Engine (Phase 1):** Finalize the logic simulator, UI shell, and ensure polished editing/simulation workflows. Complete unit tests on `rb-logic-core` and `rb-logic-view`. (Cite [20] as the current baseline vision.)
2. **Complete FPGA Bridge/Toolchain (Phase 2):** Finish `rb-fpga-bridge` REST API and test it with Basys3/Spartan boards. Automate building of all fixture bitstreams. Integrate smoke tests into CI (e.g., a GitHub Action calling `smoke.ps1`). (This is already partly done ⁶.) Develop and enforce the I/O protocol in `protocol.js` so any board stream is parsed consistently.
3. **Implement Lab Pipelines:** Develop scripts that take a student-submitted circuit (ZIP of Verilog) and run the full pipeline: synthesize, program, run tests, and generate feedback. Build the contract-test harness (using golden baselines and Vitest suites). Each lab’s minimal config (`lab.json`) will guide this.

4. **Antigravity Workflows (Phase 3):** Integrate Google's Antigravity into the developer environment. Define specific agent "missions" (e.g. "run all lab simulations", "update documentation"). Create templates for agents to produce reproducible results. Use agent tests as part of CI: e.g. `pnpm agent:verify` as shown in the production checklist ¹⁷.
5. **Website Relaunch:** Redesign the RedByte site UI to include new sections for Labs, Courses, and AI/Agent features. Migrate docs to the updated layout, ensuring the architecture and onboarding pages reflect the platform's breadth. Continuously deploy via the existing Cloudflare workflow ³.
6. **Hardware Expansion (Phase 4):** Incrementally add new FPGA boards. Begin with popular educational kits (e.g. Zybo, Arty). For each, create board-model entries, generate new pinmaps, and extend smoke tests. Use Antigravity agents to automate building and verifying demos on each new board.
7. **Feedback & AI Tutoring (Phase 5):** Develop agent/LLM modules that analyze student outputs and produce hints or grades. For example, an agent could call a ChatGPT-based service to "explain the error" when a lab fails its test. Build the UI for students to interact with these agent helpers.
8. **Continuous Improvement:** Maintain strict release processes (see the Release Checklist ¹⁸). Ensure all code merges are followed by lints/tests, and new versions are tagged and deployed. Use Contracts ("practice tests") and agent audits to catch regressions early.

Throughout, we will heavily document the process and outcomes: publish design docs (as in `AI_STATE.md`) that capture decisions and invariants. Use Cloud infrastructure to run agents at scale (e.g. VM instances for FPGA builds), and keep security in mind (isolating student code, sandboxing agent executions).

Conclusion

By combining a robust **simulation-and-hardware engine** with **AI-driven automation**, RedByte can become the go-to open platform for digital logic education. Its **modular architecture** allows any circuit to flow from the browser to real FPGA hardware seamlessly. The adoption of Google Antigravity means future development (and teaching) will be **agent-assisted**: tedious tasks like test generation, GUI tweaks, or grading can be offloaded to intelligent agents ¹¹ ¹³. Scaling beyond two boards is straightforward thanks to the abstracted board-model system ⁷. And by leveraging multiple LLMs (Gemini, Claude, GPT) within Antigravity, RedByte ensures that "closing the loop from IDE to hardware" also includes *an IDE that listens*: students get immediate AI feedback, and educators get powerful tools to expand labs and curricula.

Sources: RedByte's GitHub (architecture docs, code modules, smoke tests) ¹ ² ⁶ ⁷, and Google's Antigravity documentation ⁹ ¹¹. These guided the above architecture diagrams and development strategy.

¹ ² ¹⁵ `index.md`

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/docs/index.md>

³ ¹⁶ ¹⁷ `PRODUCTION_READINESS.md`

https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/PRODUCTION_READINESS.md

⁴ ⁵ `rb-fpga-toolchain.mjs`

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/tools/toolchain/rb-fpga-toolchain.mjs>

6 **smoke.ps1**

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/tools/smoke/smoke.ps1>

7 **discovery.js**

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/packages/rb-fpga-bridge/src/discovery.js>

8 **check-golden.js**

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/packages/rb-fpga-bridge/scripts/check-golden.js>

9 10 11 12 13 14 **Build with Google Antigravity, our new agentic development platform - Google Developers Blog**

<https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>

18 **RELEASE.md**

<https://github.com/swaggyp52/redbyte-ui-genesis/blob/31e6f66e83e5fd0d2f604e3a3209a2b1761e4134/RELEASE.md>